

NN-Classifier Integration in Solver_RG — Architecture

Description

Draft. Companion to NN_CLASSIFIER_INTEGRATION_PLAN.md; grounds claims in Solver_train/docs/SCIENTIFIC_DESCRIPTION.md and the Solver_RG source tree.

1. Background

Solver_RG is a Java-based reasoning engine. Its semantic model is an eight-member `ConceptType` enum — `COMPOSITE`, `AR1`, `PRIMITIVE`, `NUCLEUS`, `SET`, `ACTION`, `VIRTUAL`, `USAGE`. The first six are the load-bearing semantic tiers exercised by the GA and rule engine; `VIRTUAL` and `USAGE` are bookkeeping/derived markers that this integration does not touch. A `NUCLEUS` is an atomic typed attribute (for example `FigureType` $\in$ 1..6); a `PRIMITIVE` narrows a nucleus by attaching an `IRule` `m_ownRule`. Every rule honors a single `check(int value)` contract: `RuleE` accepts a comma-list of equal values, `RuleNE` its negation, and `RuleIn` a numeric `[min, max]` interval. Runtime state lives in `Situation` — a map of `IdGroup` objects, each an `AR1`-level bag of `NucleusInstance` records carrying an `int m_value`, a `PrimitiveType`, and a back-link to its `IdGroup`. Chess semantics are encoded as bare integers, with `identifyFigure(figure,color)` hardcoding the mapping `1=p`, `.6=k` and `0=empty`.

Solver_train supplies two neural network families that consume board crops:

Family	Input	Output	Backbone / head
Piece classifier (two heads)	128×128 RGB square crop	6-class type + 2-class color	ResNet-18 / EfficientNet-B0 with replaced classification head; checkpoints <code>type_classifier.pt</code> , <code>color_classifier.pt</code> carrying <code>arch</code> , <code>target</code> , <code>class_names</code> , <code>img_size</code> , <code>state_dict</code> , <code>best_val_acc</code>
Siamese neighborhood detector	two 64×64 crops	2-class neighbor / not-neighbor	shared ResNet-18; 4-way relation head <code>[f_A; f_B; f_A-f_B ; f_A*f_B]</code>

All models share an ImageNet normalization preset and expose a `class_names` / metadata contract intended for downstream consumers (`SCIENTIFIC_DESCRIPTION.md`).

2. Problem statement

Authoring rules against a categorical nucleus previously required raw integer literals — `= 6` for *king*, `= 1` for *white*. Three concrete problems follow:

- **P1. UX opacity.** The integer-to-class mapping is undocumented inside the editor; it lives implicitly in `Situation.identifyFigure()` and in training-time folder naming (`wP/`, `bQ/`).
- **P2. Encoding coupling.** Author intent ("king") is bound to a numeric encoding the author cannot see, so any rule on disk is only as portable as its current integer layout.
- **P3. Retraining drift.** If a classifier is retrained with reordered `class_names`, every previously authored rule silently flips meaning, with no detectable error.

The integration replaces the *surface* with the NN's own `class_names` while leaving the *engine* — `check(int)`, GA conjunction, instance storage — operating on integers.

3. Architecture

3.1 Vocabulary abstraction

`Vocabulary` is an immutable name↔index bridge with no knowledge of concepts or rules. It owns an ordered `classNames` list plus a bidirectional map, and exposes `indexOf(String)` and `nameOf(int)`. The full manifest field set it consumes is documented once in §3.3. `PrimitiveType` gains an optional `m_vocabulary` field: numeric-only types (`cordX`, `cordY`) leave it null; categorical types bind to a registered `Vocabulary`. The abstraction is deliberately one-way — a `Vocabulary` exposes ints to the engine and names to the UI, and nothing else changes shape.

3.2 RuleIs / NOT IS operators

`RuleIs` stores one or more symbolic names and resolves them at construction time against the bound `Vocabulary`. Its `check(int)` defers to a plain integer comparison, so the GA / rule pipeline observes no behavioral change. Multi-symbol values (e.g. "knight, bishop" to express *minor piece*) parse as comma-lists with OR semantics; the negated form is produced by `RuleIs.createNegated()`, with no separate `RuleNotIs` source. Conjunction with an inherited parent rule projects symbolic names to an integer set, delegates to the integer-native conjunction in `RuleE` / `RuleNE`, and re-symbolizes the result — an adapter layer, not a rewrite. Operator strings `IS` and `NOT IS` are registered as `Definitions.isOperator` / `notIsOperator` and wired into `ConceptCreator.createRule()`, which rejects either operator unless a non-null `Vocabulary` is supplied.

3.3 Dynamic classifier discovery — canonical manifest schema

A `VocabularyRegistry` singleton scans a drop folder at startup. Directory resolution precedence (first hit wins):

1. JVM system property `solver.classifiers.dir`
2. environment variable `SOLVER_CLASSIFIERS_DIR`
3. `./classifiers` under the process working directory

Each subfolder containing a `manifest.json` is registered. The manifest schema is authoritative wherever it is referenced elsewhere in this document:

- **schemaVersion** — required, 1.
- **classifierId** — required, stable primary key.
- **kind** — required, one of `categorical`, `relation`, `regression`.
- **classNames** — required for `categorical`; ordered, authoritative for the index mapping.
- **displayName** — optional, human label.
- **suggestedPrimitiveTypeName** — optional, hint used for indexing by nucleus type.
- **checkpointPath** — optional, resolved relative to the manifest.
- **inputSpec**, **trainingMetadata** — optional, opaque to the engine; reserved for drift-detection tooling.

The registry indexes by `classifierId` and by `suggestedPrimitiveTypeName`; tests can seed via `registerForTest()`. Malformed manifests are logged and skipped without blocking startup. Non-categorical manifests (`relation`, `regression`) receive a placeholder `"__noncategorical__"` entry so they remain visible to `listAll()` but are flagged non-pluggable (see §3.5).

Container wiring: the Dockerfile exports `solver.classifiers.dir=/opt/solver/classifiers` into `CATALINA_OPTS` (Dockerfile); `docker-compose.yml` bind-mounts the host `./classifiers` folder read-only into the container (docker-compose.yml).

3.4 Authoring surface (UI architecture)

The authoring surface is the second half of the architecture, separated from the engine by the Struts action layer.

- **Nucleus editor.** `nucContent.html` adds `IS / NOT IS` to the operator `<select>`, and a `<select class="f_classifier">` plus a hidden `.f_symbolValue` multi-select next to the value-attribute row.
- **Multi-select with `[all]` / `[none]` / `[apply]`.** A nucleus defaults to "all classes selected" (natural reading: *any output of the classifier*); a primitive starts empty so the author pins specific classes (`nucConcept.js`). `commitSymbolSelection()` joins the chosen names into a comma string into `.f_symbolValueCommitted`, and the existing save path then sees a plain value field.
- **Primitive editor mirror.** `primConcept.js` reads classifier + operator, commits the symbol state, and emits a `classifier` JSON key alongside the symbolic value.
- **Struts endpoints.** `ListClassifiers` returns every registered classifier with a `pluggable` flag; `GetVocabulary` returns the full ordered `classNames` for a given `classifierId`. The parser is tolerant of whitespace, case, and either comma-list or JSON-array payloads.

3.5 Projected image-to-situation pipeline

The pipeline below is described as the intended end-to-end flow; §5 is the canonical status reference.

- **Image capture / upload.** A board image arrives via the existing situation-creation UI.
- **Python inference sidecar.** A separate process loads each `.pt` checkpoint from the directory the manifest's `checkpointPath` resolves to, runs the type and color CNNs over per-square 128×128 crops, and emits, per cell, `(classifierId, name, prob)` tuples.
- **Optional Siamese verification.** The neighborhood detector validates the 8×8 grid geometry before commit. The detector is `kind:relation` and so non-pluggable to the nucleus picker (see §6).
- **Server-side bridge.** A `SituationCreator.createSituationFromImage()` hook consumes sidecar output and, for each detected cell, creates `NucleusInstance` records inside an `IdGroup` keyed by `(cordX, cordY)`, setting `m_value = VocabularyRegistry.get(classifierId).indexOf(name)`.
- **Empty-square / occupancy gap.** The piece type CNN was trained on pieces only — there is no *empty* class. To preserve the legacy `FigureType` mapping where `0=empty`, every categorical vocabulary that needs occupancy semantics prefixes a synthetic `"empty"` entry at index 0. An upstream occupancy step is required before piece classification can be trusted.

4. Validation

Three JUnit suites cover the delivered surface:

- `TestVocabulary` covers round-trip `indexOf / nameOf`, unknown-name rejection, duplicate-class rejection, empty-list rejection, and registry seeding by id and by suggested primitive-type name.
- `TestRuleIs` covers `IS` against a single name, `NOT IS` inversion, multi-symbol membership (`"knight, bishop"`), unknown-symbol rejection at construction, clone fidelity, JSON emission containing the classifier id, and conjunction with `RuleE`.
- `TestRegistrySmoke` is an ad-hoc `main` used to verify that the drop folder is picked up at startup.

End-to-end evaluation against a live image pipeline is deferred to Phase D (see §5).

5. Implementation status

Status uses three values: **Delivered**, **Projected** (planned, partially scaffolded), **Out of scope** (explicit non-goal of this integration).

Component	Status	Notes
Vocabulary + VocabularyRegistry + drop-folder scan	Delivered	§3.1, §3.3
PrimitiveType.m_vocabulary binding	Delivered	§3.1
RuleIs / NOT IS + Definitions wiring + ConceptCreator branch	Delivered	§3.2
ListClassifiers + GetVocabulary Struts actions	Delivered	§3.4
Nucleus / primitive editor UI (picker, multi-select, mirror input, preview)	Delivered	§3.4
Docker env var + bind mount	Delivered	§3.3
Symbolic JSON on save ({oper, value, classifier})	Delivered	RuleIs.getJSON
manifest.json contract documented	Delivered	§3.3
JUnit suites (vocab, rule, registry)	Delivered	§4
Python inference sidecar	Projected	Phase D
SituationCreator.createSituationFromImage()	Projected	Stub only
Siamese verification step	Projected	Non-pluggable to nucleus picker
Confidence-score propagation to NucleusInstance	Out of scope	argmax only
Regression-NN support (RuleRange(float))	Out of scope	Would need new instance value type
Auto-migration on classifier retrain	Out of scope	Detection only, via trainingMetadata

6. Discussion

Why the rule engine remained integer-based. The GA pipeline, conjunction reduction, and `Situation.applyAction()` all assume `check(int)` over a dense small-int domain. Re-typing the

engine to carry strings would have required rewriting `RuleE`, `RuleIn`, dependency resolution in `setDependency()`, and every serialized rule on disk. The chosen design keeps the engine integer-native and inserts a `Vocabulary` adapter at a small, enumerable seam set: (i) the `PrimitiveType.m_vocabulary` field, (ii) `VocabularyRegistry` bootstrap, (iii) the `ConceptCreator.createRule()` JSON resolver, (iv) the `RuleIs.getJSON` emit path, and (v) the `ListClassifiers / GetVocabulary` Struts actions. GA throughput, action application, and persisted rule semantics are unchanged.

Design envelope. The abstraction generalizes to any new domain whose nucleus is categorical — traffic-sign types, terrain classes, X/Y coordinate classifiers treated as small categorical vocabularies. It does not generalize to regression NNs (angle, bounding-box, threshold), which would need a new `RuleRange(float)` plus a widened instance value type and are explicitly out of scope. Relational NNs (the Siamese neighborhood detector is the canonical example) are listed in `ListClassifiers` for visibility but cannot bind to a nucleus rule: a relation is an AR1-level composite predicate, not a `check(int)` over one instance; a composite-rule integration surface is left as future work.

Design rationale. Three local decisions deserve explicit justification. *Drop folder over DB-backed catalog.* A filesystem manifest keeps the trust boundary at the container's read-only mount, requires no schema migration on retrain, and matches how `Solver_train` ships artifacts today; a database catalog would add a write path orthogonal to the engine. *JSON manifest over a Python-side gRPC handshake.* The handshake would couple `Solver_RG` startup to the inference sidecar's availability; a static manifest lets the engine come up, expose the authoring surface, and tolerate a missing sidecar (`P1/P2` are addressed without the perception layer running). *Argmax over confidence-thresholded labels.* Propagating probabilities to `NucleusInstance` would widen its value type and ripple through `check(int)`; argmax preserves the integer engine and keeps confidence as a future per-instance side-channel.

Drift risk (P3). A retrained NN whose `class_names` reorders would silently invert any previously stored integer-valued rule. The symbolic JSON emission turns this from a silent corruption into a detectable mismatch: the manifest is authoritative for the current index mapping, and each persisted rule's `value` is a name that can be re-resolved on load. The system does *not* auto-migrate persisted instance integers; `trainingMetadata.producer / timestamp` is carried in the manifest precisely so a drift check can be wired in later.

A block diagram showing the parallel data paths (image → sidecar → server bridge → `IdGroup` / `NucleusInstance` → rule check) and authoring paths (UI → `ListClassifiers / GetVocabulary` → `RuleIs` → JSON persistence) is forthcoming.